COMPTE-RENDU FINAL – PROJET FIL ROUGE CYBERSÉCURITÉ

---

1. Introduction

Dans le cadre du projet fil rouge de cybersécurité, notre groupe a développé une application web simple, l'a déployée dans une machine virtuelle Linux, puis a étudié plusieurs vulnérabilités courantes.

La suite du projet consistait également à **configurer et sécuriser l'environnement serveur**, afin de comprendre comment limiter les risques identifiés.

Ce compte-rendu présente :

1. Le développement de l'application web
2. L'installation dans une machine virtuelle
3. La démonstration d'attaques (SQLi, XSS, CSRF, MITM)
4. Les travaux de configuration et de sécurisation du serveur (Projet Sevan)

---

2. Phase 1 – Développement de l'application

Nous avons conçu une application web permettant de publier et d'afficher des commentaires.

Le projet est composé des fichiers suivants :

- **index.html** → formulaire d'ajout d'un commentaire
- **submit.php** → récupération des données et insertion en base
- **comments.php** → affichage de tous les commentaires
- **sql/init.sql** → création de la base et de la table MySQL

Le site est volontairement vulnérable :

aucune protection n’est appliquée aux entrées utilisateur (pas de validation, pas d’échappement, pas de filtrage).

Cela permet d’observer clairement comment les attaques fonctionnent.

---

3. Phase 2 – Hébergement dans une Machine Virtuelle

Nous avons déployé l’application dans une VM Linux (Debian/Ubuntu) afin de reproduire un environnement réel d’hébergement.

Travaux réalisés :

- installation d’Apache (ou serveur PHP intégré)
- installation de PHP + MySQL / MariaDB
- exécution du script **init.sql** pour créer la base
- transfert du dossier **tp-comments** dans la VM
- lancement du serveur web
- accès à l’application via l’adresse IP de la VM

L’isolation de la VM permet de tester des attaques sans risquer d’impacter le réseau de l’école.

---

4. Phase 3 – Tests d'attaques

Nous avons testé quatre attaques majeures sur la version non sécurisée du site.

4.1 – Injection SQL

Cause : concaténation directe des entrées utilisateur dans les requêtes SQL.

Exemple testé :

```
'); INSERT INTO comments (content) VALUES ('Injection SQL réussie'); --
```

Résultat :

Exécution de requêtes non prévues → modification de la base.

4.2 – XSS (Cross-Site Scripting)

Cause : absence d'échappement lors de l'affichage.

Exemple testé :

```
<script>alert("XSS")</script>
```

Résultat :

Exécution de JavaScript → preuve que la faille XSS est exploitable.

4.3 – CSRF (Cross-Site Request Forgery)

Cause : absence de token CSRF dans le formulaire.

Principe du test :

Une page externe envoie un POST automatique à *submit.php*.

Résultat :

Le commentaire est accepté → preuve qu'un attaquant peut agir à la place de l'utilisateur.

4.4 – MITM (Man-In-The-Middle)

Cause : communication en HTTP non chiffrée.

Test : capture réseau interne à la VM.

Résultat :

Les données (dont les commentaires) apparaissent en clair → aucune confidentialité.

---

5. Phase 4 – Configuration et Sécurisation du Serveur

(Travaux du Projet Sevan)

5.1 Isolation, Utilisateurs et Accès

- L'application est isolée dans une machine dédiée.
- Création de l'utilisateur administrateur **sevan** :

```
adduser sevan
usermod -aG sudo sevan
```

Sécurisation MySQL

Création d'un utilisateur dédié **fil_rouge**, limité à la base **tp_comments** :

```
CREATE USER 'fil_rouge'@'localhost' IDENTIFIED BY 'root';
GRANT SELECT, INSERT, UPDATE, DELETE ON tp_comments.* TO 'fil_roug
e'@'localhost';
FLUSH PRIVILEGES;
```

Objectif : appliquer le principe du **moindre privilège**.

5.2 Sécurité Réseau et Web

Configuration du Pare-feu UFW

Seuls les ports nécessaires sont autorisés :

```
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
```

Tous les autres ports restent bloqués par défaut.

Mise en place de HTTPS (Certbot)

Installation :

```
apt install certbot python3-certbot-apache
certbot --apache
```

Le certificat Let's Encrypt est installé automatiquement et la redirection HTTP → HTTPS est configurée.

Web Application Firewall – ModSecurity

Installation :

```
apt install libapache2-mod-security2
```

Activation du moteur de blocage :

Dans le fichier de configuration :

```
SecRuleEngine On
```

Puis redémarrage :

```
systemctl restart apache2
```

ModSecurity est configuré avec l'ensemble de règles **OWASP CRS**.

5.3 Sauvegarde et Maintenance

Script de sauvegarde automatique

Chemin du script :

`/usr/local/bin/sauvegarde_finale_sevan.sh`

Contenu :

```
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
LOCAL_BACKUP_TARGET="/var/backups_locales"
DAYS_TO_KEEP=7

HOME_PATH="/home/sevan"
CONFIG_PATH="/etc"
```

Rapport sécurité appliqué Côté SLAM

Le **sécurité appliquée** (les modifications apportées au code) côté SLAM sont les suivantes :

1. Sécurité Anti-injection SQL : préparation des requêtes SQL
2. Sécurité Anti-XSS : application des caractères spéciaux pour tout texte saisi et envoyé depuis le formulaire.
3. Sécurité Anti-CSRF : simulation de token de session vérifiée côté backend (PHP)

I) Sécurité Anti-injection SQL

La sécurité anti-injection SQL consiste à préparer une requête côté backend afin d'empêcher toute intrusion suspecte, susceptible de nuire à l'intégrité des données.

Code avant : sans requête préparée

```
try { $pdo = new PDO($dsn, $user, $pass);

$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $pdo->query("SELECT id, content, created_at FROM comments ORDER BY created_at DESC");

$comments = $stmt->fetchAll(PDO::FETCH_ASSOC); }

catch (PDOException $e) { echo "Erreur SQL : " . $e->getMessage();

exit;

 }
```

Code après : avec requête préparée

```
 try {

     $pdo = new PDO($dsn, $user, $pass);

     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

     $sql = "INSERT INTO comments (content) values (?)";

     $stmt = $pdo->prepare($sql);

     $stmt->execute([$content]);

     header('Location: comments.php');

     exit;

  } catch (PDOException $e) {
```

```
    echo "Erreur SQL : " . $e->getMessage();
  }
```

II) Sécurité Anti-XSS

La sécurité Anti-XSS consiste à apporter des correctifs à son code côté client en spécifiant les types de données à recevoir. Cela permet au navigateur de les lire comme du texte brut plutôt que comme un script. Elle permet de se prémunir de plusieurs risques, notamment le vol des cookies de connexion.

Code avant : sans htmlspecialchars

```
<?php foreach ($comments as $c): ?>
<li>
<div><?= $c['created_at'] ?></div>
<div><?= $c['content'] ?></div>
</li>
<?php endforeach; ?>
</ul>
<?php endif; ?>
```

Code après : avec htmlspecialchars

```
<ul>
 <?php foreach ($comments as $c): ?>
 <li>
 <div><?= htmlspecialchars($c['created_at'] )?></div>
 <div><?= htmlspecialchars($c['content']) ?></div>
 </li>
 <?php endforeach; ?>
 </ul>
```

III) Sécurité Anti-CSRF

La sécurité Anti-CSRF consiste en la mise en place d'un cookie de session unique généré automatiquement lors de l'envoi d'un formulaire. Elle permet de protéger l'utilisateur contre les cas d'usurpation.

Code avant : sans CSRF

Index.html :

```
<!DOCTYPE html>
<html lang="fr">
 <head>
  <meta charset="utf-8" />
  <title>Poster un commentaire (vulnérable)</title>
  <link rel="stylesheet" href="style.css" />
  <link rel="icon" type="image/gif" href="favicon.png" />
 </head>
 <header class="hd-ent"><strong>HackingMotions</strong></header>
 <body>
  <h1>Poster un commentaire</h1>
  <form method="post" action="submit.php">
   <label
    >Message<br />
    <textarea name="content" rows="5" cols="60" required></textarea></label
   ><br />

   <button type="submit">Envoyer</button>
  </form>
  <p>
   Après envoi, vous serez redirigé vers la page qui affiche tous les
   commentaires.
```

```
  </p>
 </body>
</html>
```

Submit.php :

```
<?php
$host = '127.0.0.1';
$db   = 'tp_comments';
$user = 'fil_rouge';
$pass = '#'; // adapter si besoin
$charset = 'utf8mb4';
$dsn = "mysql:host=$host;dbname=$db;charset=$charset";



$content = $_POST['content'] ?? '';
if(!empty([$content] && !empty([$userSecure])){
  try {
    $pdo = new PDO($dsn, $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $sql = "INSERT INTO comments (content) values (?)";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$content]);
    header('Location: comments.php');
    exit;
  } catch (PDOException $e) {
    echo "Erreur SQL : " . $e->getMessage();
  }
} else {
  header('Location: index.html');
```

```
  exit;
}
?>
```

Code après : avec CSRF

Index.html :

```
<!DOCTYPE html>
<html lang="fr">
 <head>
  <meta charset="utf-8" />
  <title>Poster un commentaire (vulnérable)</title>
  <link rel="stylesheet" href="style.css" />
  <link rel="icon" type="image/gif" href="favicon.png" />
 </head>
 <header class="hd-ent"><strong>HackingMotions</strong></header>
 <body>
  <h1>Poster un commentaire</h1>
  <form method="post" action="submit.php">
   <label
    >Message<br />
    <textarea name="content" rows="5" cols="60" required></textarea></label
   ><br />
   <input
    type="text"
    name="token_csrf"
    value="123.csrf"
    class="hidden-token"
   />
```

```
    <button type="submit">Envoyer</button>
   </form>
   <p>
    Après envoi, vous serez redirigé vers la page qui affiche tous les
    commentaires.
   </p>
  </body>
</html>
```

Submit.php :

```
<?php
$host = '127.0.0.1';
$db   = 'tp_comments';
$user = 'fil_rouge';
$pass = '#'; // adapter si besoin
$charset = 'utf8mb4';
$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
if ($_SERVER['REQUEST_METHOD'] === 'POST')
{  $userSecure=$_POST['token_csrf'] ?? '';
   if ($userSecure !== '123.csrf' ){
      die('mauvais token.');
   }


$content = $_POST['content'] ?? '';
if(!empty([$content] && !empty([$userSecure])){
   try {
      $pdo = new PDO($dsn, $user, $pass);
      $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
      $sql = "INSERT INTO comments (content) values (?)";
```

```
        $stmt = $pdo->prepare($sql);
        $stmt->execute([$content]);
        header('Location: comments.php');
        exit;
    } catch (PDOException $e) {
        echo "Erreur SQL : " . $e->getMessage();
    }
  } else {
    header('Location: index.html');
    exit;
  }


}
?>
```

Merci d’avoir lu la documentation jusqu’au bout !
Tâchez de mettre tout ce qui a été dit en pratique pour une code base plus résiliente 👌

```
# Configuration DB
DB_USER="fil_rouge"
DB_PASS="root"
DB_NAME="tp_comments"

# Sauvegardes
tar -czf $LOCAL_BACKUP_TARGET/home_sevan_$DATE.tar.gz $HOME_PAT
H
tar -czf $LOCAL_BACKUP_TARGET/configs_$DATE.tar.gz --exclude={'/etc/
apt/*','/etc/ssh/*','/etc/ssl/private/*'} $CONFIG_PATH
mysqldump -u $DB_USER -p$DB_PASS $DB_NAME | gzip > $LOCAL_BACK
UP_TARGET/db_dump_$DATE.sql.gz

# Nettoyage (+7 jours)
find $LOCAL_BACKUP_TARGET -type f -name "*.gz" -mtime +$DAYS_TO_K
EEP -delete

exit 0
```

Planification (cron)

Rendre exécutable :

```
chmod +x /usr/local/bin/sauvegarde_finale_sevan.sh
```

Tâche planifiée dans la crontab root :

```
30 3 * * * /usr/local/bin/sauvegarde_finale_sevan.sh
```

Backup automatique chaque jour à **3h30**.

---

6. Conclusion

Ce projet nous a permis :

- de comprendre la construction d’une application web vulnérable,
- de l’héberger dans un environnement isolé,
- de découvrir et exploiter quatre failles majeures (SQLi, XSS, CSRF, MITM),
- d’apprendre les bases de la sécurisation serveur : utilisateurs, pare-feu, HTTPS, WAF, sauvegardes.

Les phases suivantes porteront sur le durcissement complet de l’application (SLAM) ou de l’infrastructure (SISR).